

Bird Species Classification Using CNN

CNN-based fine-grained image classification for bird species

Team Number: Group 02

Team Members: Eunseo Seo, Minjun Park, Gun Kim,
Chaemin Lee

Dataset: 200-class bird image classification task



Project Goal

Why this problem matters

- The number of objects from each class are all different.
- Even though some images are from a same class, they can show different poses, scales, or backgrounds.
- A practical model should return top-3 predictions and confidence scores, not only one label.

Core output

Predicted species + confidence score + top-3 candidate classes

Dataset

200 bird classes
Train: 5,994
Validation: 2,897
Test: 2,897

Models

Baseline CNN
ResNet
EfficientNetV2

Demo

Upload image →
Top-3 predictions

Assignment alignment

- Implement a custom baseline CNN (model 1) before moving to the improved model (model 2).
- Test meaningful improvements such as augmentation, dropout, and learning-rate tuning.
- Present learning curves, confusion matrices, and a lightweight inference demo.

Dataset Overview

200

Classes

5,994

Train images

2,897

Validation images

2,897

Test images

Project split used in the bird experiments

Train 5,994

Val 2,897

Test 2,897

Fine-grained setting

Bird species can share overall color and shape, so identifying small details is the main importance.

Per-class scale

Average images/class
~30 train
~14-15 val/test

Difficulty

High class count and variation of pose and background make the task harder.

Image Classification Challenge



Complex background



Different pose



Different age



With other objects

Main difficulty

Inter-class differences such as background, pose, and interruption from other objects can make classification more difficult.

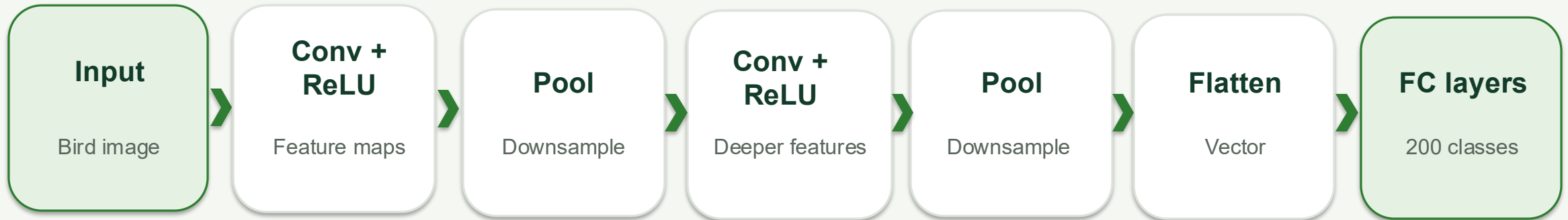
What the model needs

Classification texture, color, contour, and local-part features that stay useful across poses, backgrounds, and scales.

How to improve the model

Data augmentation, tuning parameters, and using pretrained backbones can help improving our CNN model.

Model 1: Baseline CNN



Model 1: Baseline CNN training result

48.8%

Final test accuracy

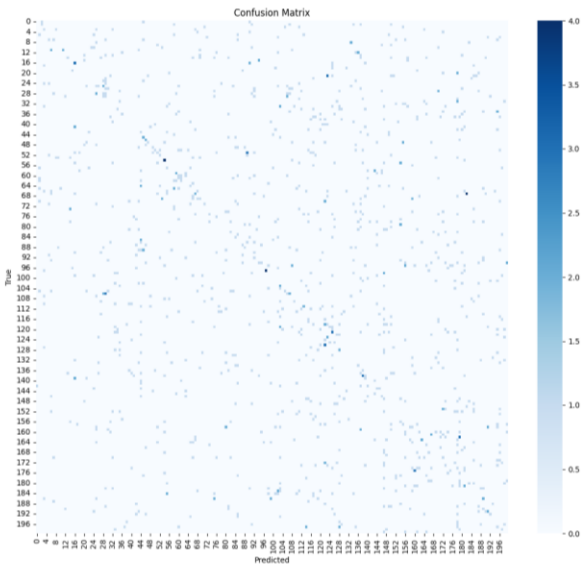
~100%

Train accuracy
near the end

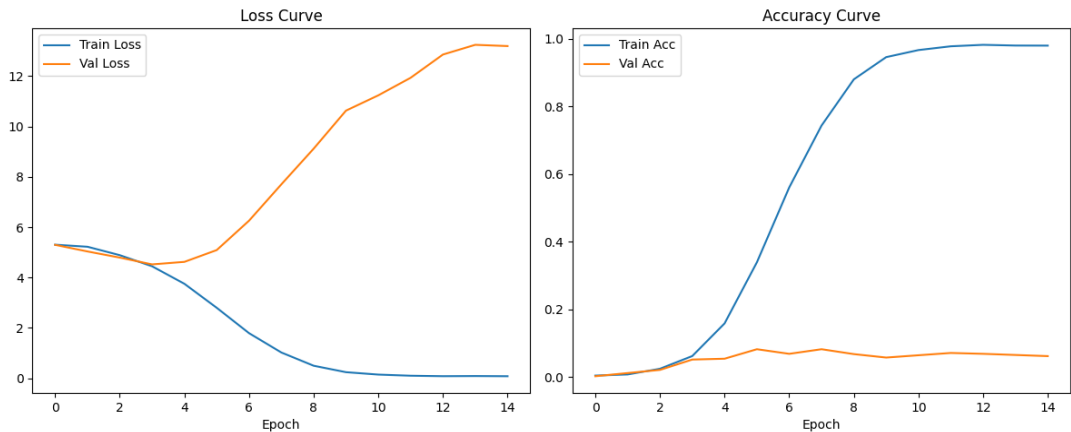
Large gap

Strong overfitting signal

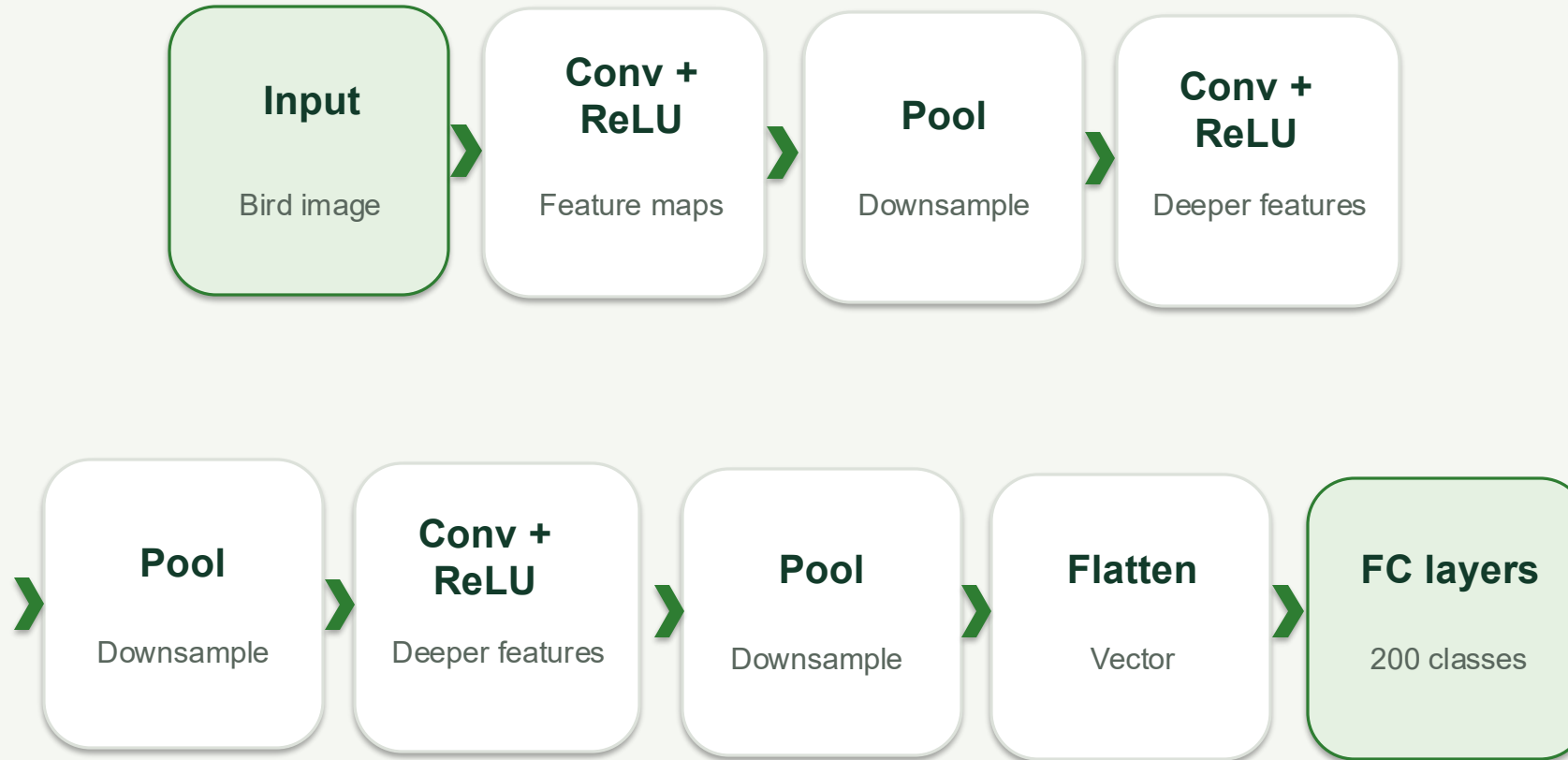
Confusion matrix



Learning curves



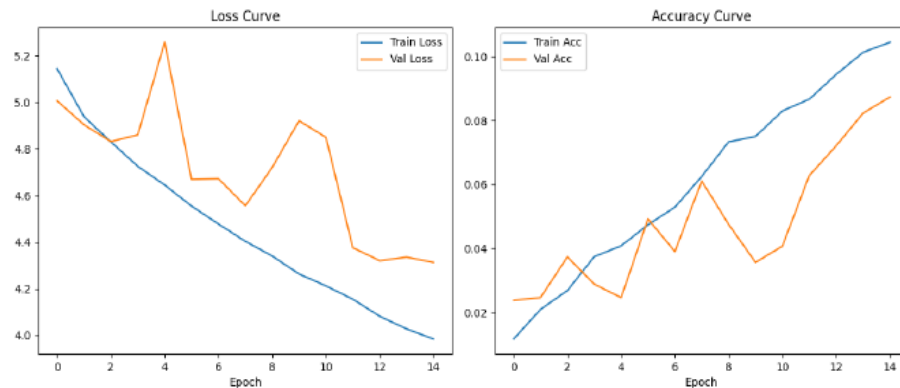
Model 1: Baseline CNN with 3 conv layers+dropout



Model 1: Baseline CNN with 3 conv layers+dropout

3 conv layers + dropout

Manual baseline improvement



Better trend, but still far from practical performance

What changed

The team first modified only the custom baseline because that was the model they could redesign directly.

What improved

According to the notes, loss began to decrease and validation accuracy started to rise more clearly after moving from 2 to 3 conv layers and adding dropout.

Why it still plateaued

Even with a better custom baseline, the model capacity and learned features were still too weak for 200-way fine-grained classification.

Next decision

This motivated the switch to pretrained backbones such as ResNet, and later EfficientNetV2, to learn stronger visual representations.

Model 2: ResNet

48.94%

ResNet only

50.30%

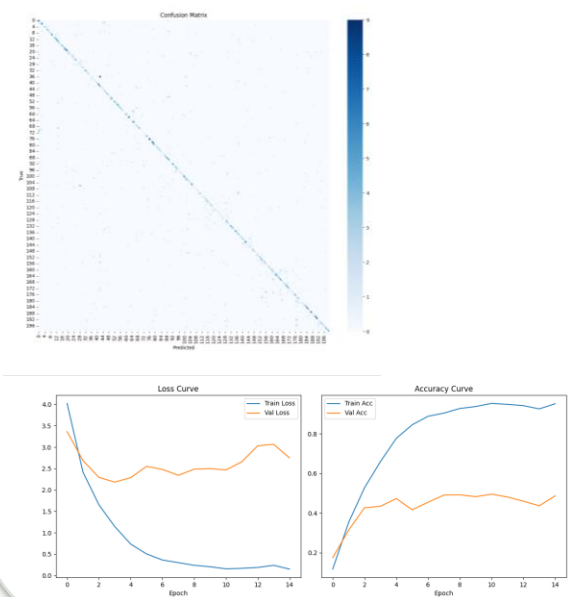
ResNet + dropout

59.53%

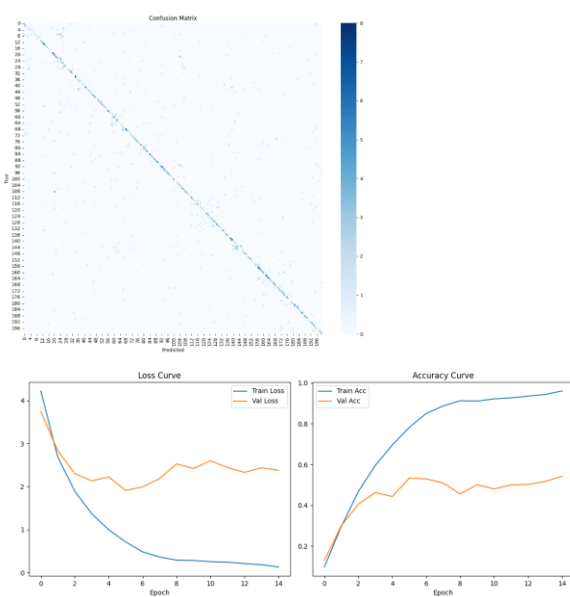
ResNet + dropout/aug/norm
/bounding box

Pretraining alone was not enough.
The gain became meaningful after augmentation and normalization.

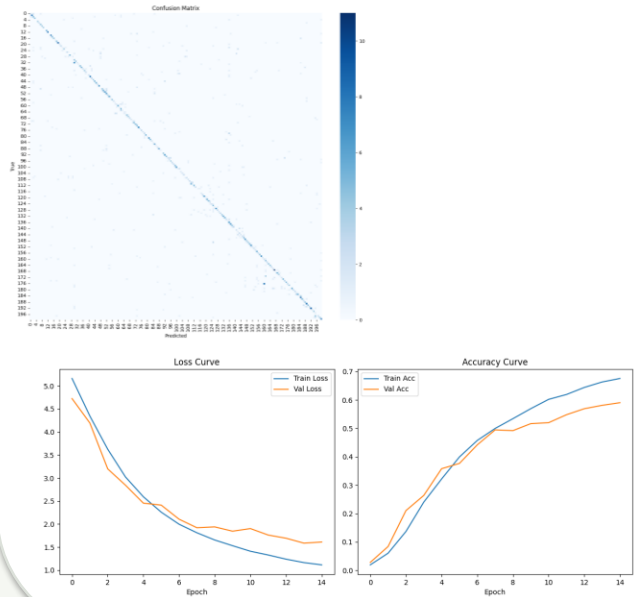
ResNet only



ResNet + dropout

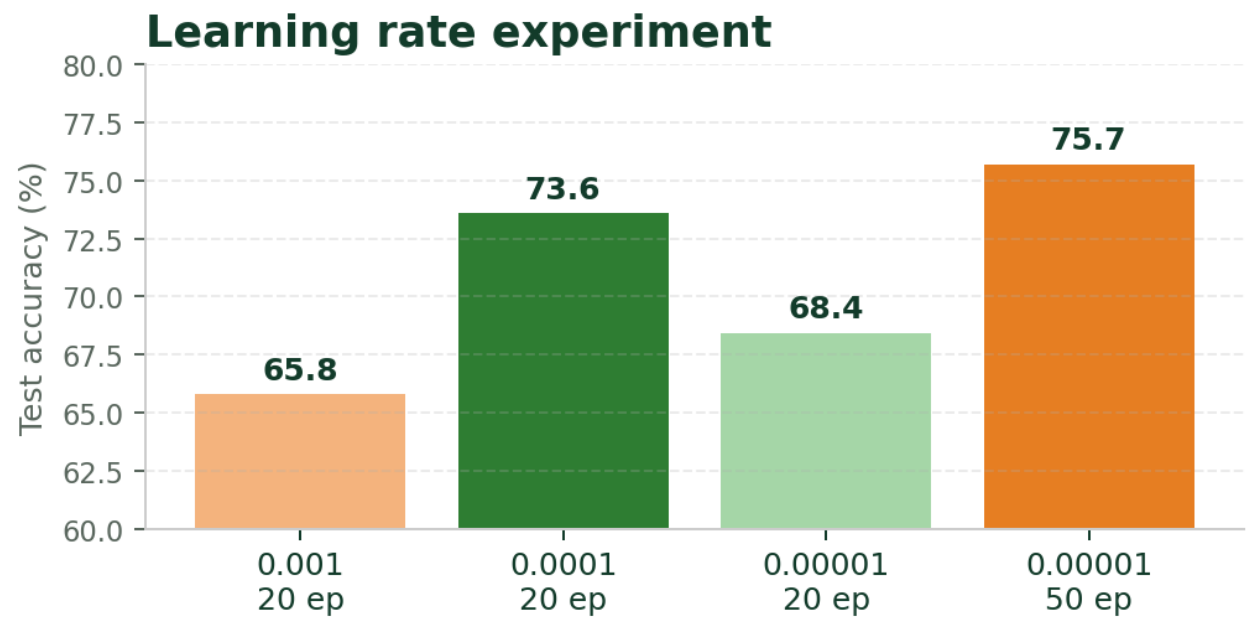


ResNet + dropout/aug/norm



Model 2: ResNet LR experiment

Accuracy across LR settings



Key lesson: schedule or longer fine-tuning matters when the LR is small.

75.7%

Best LR setting result

0.001 -> 65.79%

This setting improved over the early ResNet stage, but it likely stepped too aggressively and plateaued lower.

0.0001 -> 73.58%

A smaller LR produced a much stronger result, indicating more stable fine-tuning.

0.00001 with more epochs -> 75.7%

The very small LR needed longer training, but with 50 epochs it reached the best result among the LR trials.

Next step -> stronger architecture

The PDF lists four learning-rate settings, including a longer 50-epoch run for the smallest LR.

Model 2: Architecture Shift to EfficientNetV2

[Parameter Setting]
Learning Rate : 0.00001
Optimizer : adam
Resize: 224 X 224
Batch size : 64
No dropout

ResNet



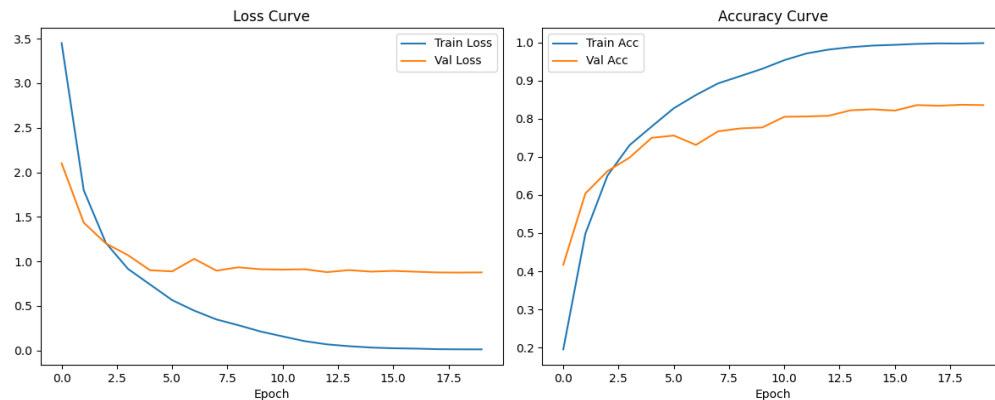
EfficientNetV2-S
+ lr Scheduler

84.5%

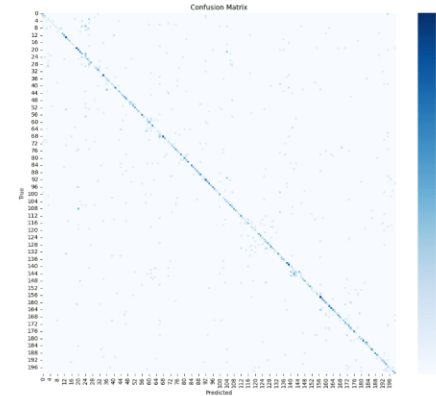
Test accuracy

Initial EfficientNetV2

Learning curves



Confusion matrix



Why Performance Improved

EfficientNet V2-S

- Uses Higher Input Resolution
- Captures Subtle Bird Features
- Improves Fine-Grained Classification Performance
- Higher Accuracy with Fewer Parameters
- Fused-MBConv
- 35sec per epoch

BUT still severe overfitting and unstable !!

Model Improvement 1: LabelSmoothing

EfficientNetV2



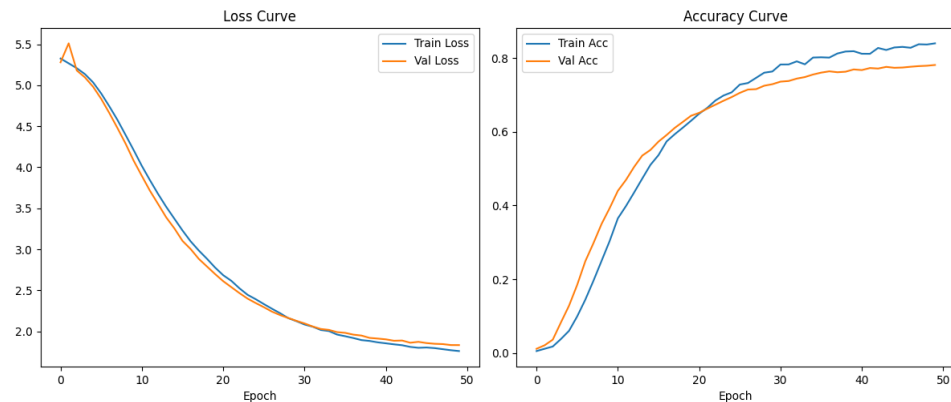
EfficientNetV2
+
Label Smoothing

77.7%

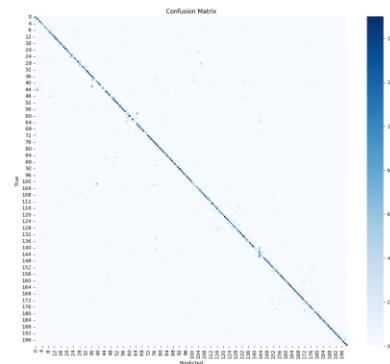
Test accuracy

LabelSmoothing 0.1

Learning curves



Confusion matrix



What is Changed?

- Label smoothing prevents confidence by forcibly assigning an error rate to the model.

Alleviated overfitting !

Model Improvement 2: Higher resolution

EfficientNetV2



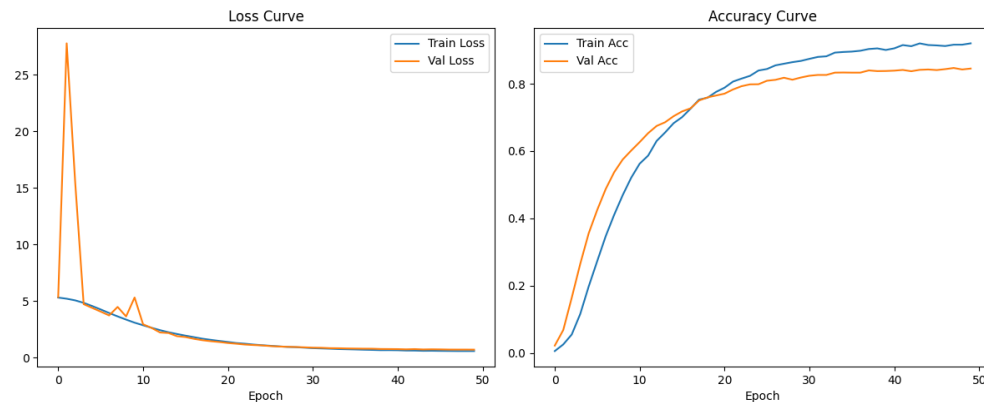
EfficientNetV2
+
Higher resolution

84.1%

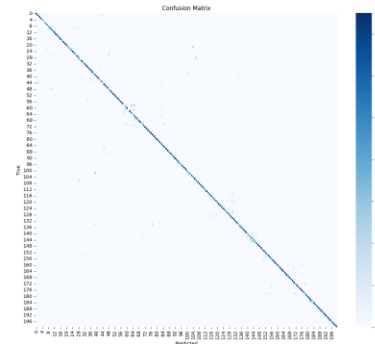
Test accuracy

EfficientNetV2+Higher resolution

Learning curves



Confusion matrix



What is Changed?

- Increasing the number of pixels in the input image enables the model to recognize subtle differences.
- Long time of study per epoch (35s -> minimum 1 minute per Epoch)

Long time per epoch

Model Improvement 3: Auto Augmentation

EfficientNetV2



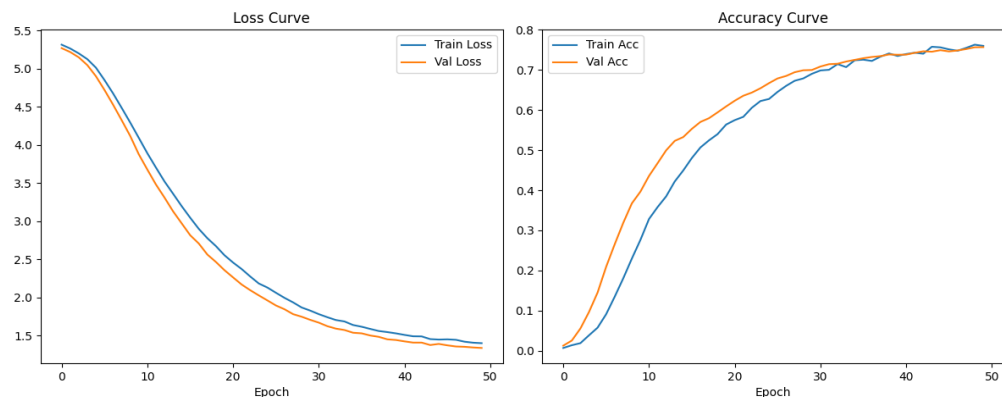
EfficientNetV2
+
auto augmentation

76.7%

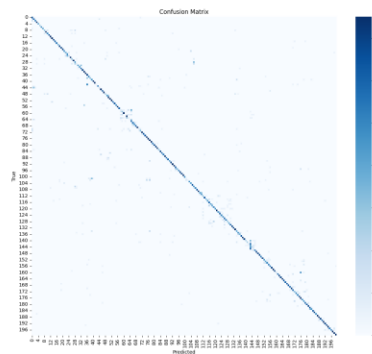
Test accuracy

auto augmentation (Randaugment)

Learning curves



Confusion matrix



What is Changed?

- Auto-augmentation transforms the image differently each time, effectively increasing the data.
(our less dataset image)

Model Improvement 4: opti-adam-weight decay 0.0001

EfficientNetV2



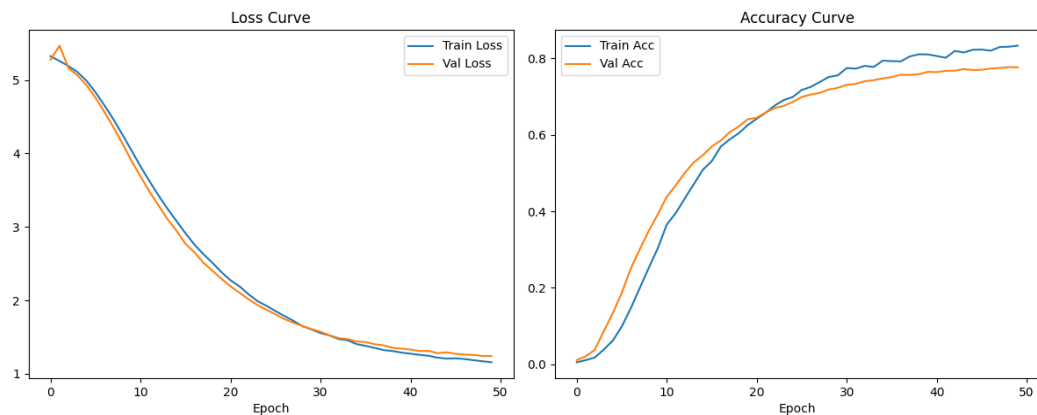
EfficientNetV2
+
opti-adam-weight decay

77.4%

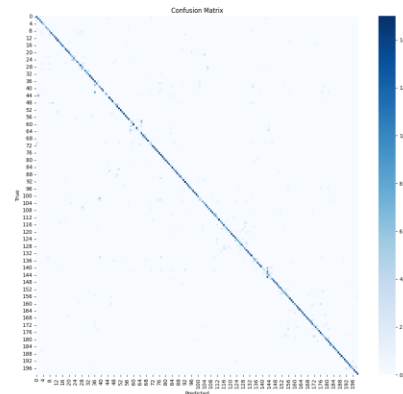
Test accuracy

opti-adam-weight decay

Learning curves



Confusion matrix



What is Changed?

- Continuously trimming a portion of the values to prevent the weights from becoming too large, thereby avoiding dependence on the training data.

Experiment Plan

Default EfficientNetV2-S

84.1% (23 epoch)

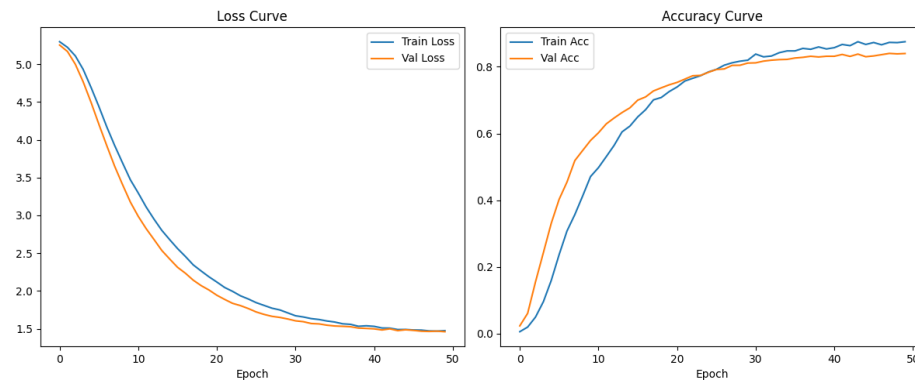
Test accuracy

[Default Parameter Setting]

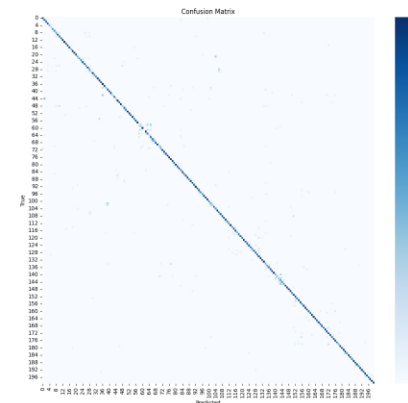
Epoch : 50
Learning Rate : 0.00001
Optimizer : adam
Resize: 384 X 384
Batch size : 32
Dropout : 0.5
Tmax = args.epochs
Eta_min: 0.000001
Weight decay: 0.0001
Label smoothing: 0.1

Default

Learning curves



Confusion matrix



What will be Changed in the following experiments?

- lr, dropout
- label smoothing
- Optimizer
- scheduler ($\eta_{\min}$, $t_{\max}$)
- weight decay...

Experiment 1: dropout 0.3

Dropout 0.5



Dropout 0.3

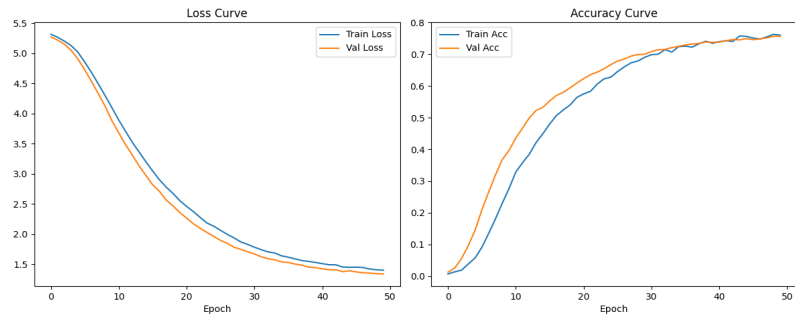
Dropout 0.7

85.5%

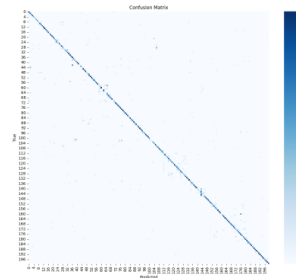
79.9%

Dropout 0.3

Learning curves

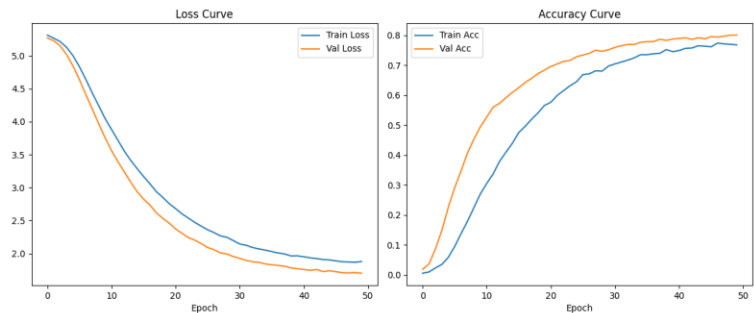


Confusion matrix

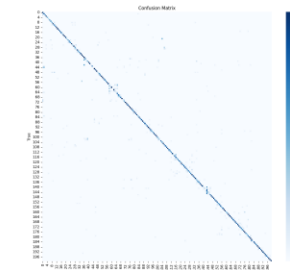


Dropout 0.7

Learning curves



Confusion matrix



[Parameter Setting]
Learning Rate : 0.00001
Optimizer : adam
Resize: 384 X 384
Batch size : 32
dropout : 0.5 -> 0.3, 0.7
Tmax = args.epochs
Eta_min: 0.000001
Weight decay: 0.0001
Label smoothing: 0.1

What is Changed?

- Dropout rate

0.3: higher accuracy and small difference between train and validation

0.7: lower accuracy, model not study fully about dataset
-> underfitting

Experiment 2: label smoothing

[Parameter Setting]
Learning Rate : 0.00001
Optimizer : adam
Resize: 384 X 384
Batch size : 32
dropout : 0.5
Tmax = args.epochs
Eta_min: 0.000001
Weight decay: 0.0001
Label smoothing: 0.1 -> 0.2

Default EfficientNetV2
label smoothing 0.1



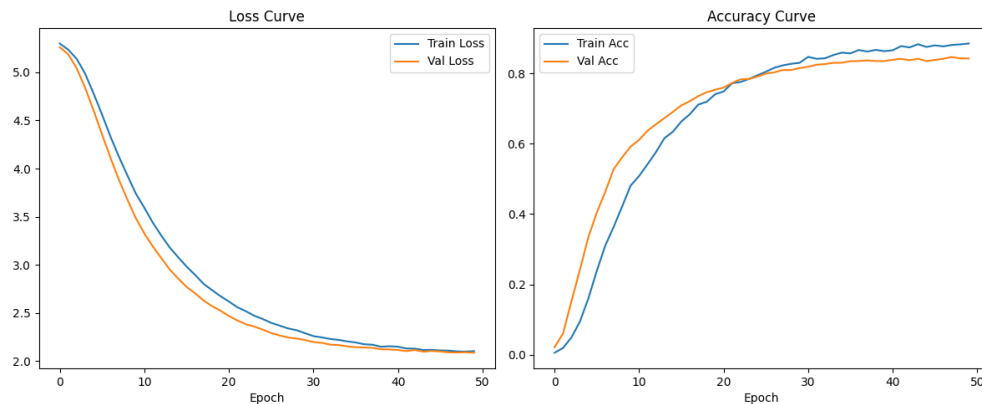
label smoothing
0.2

84.4%

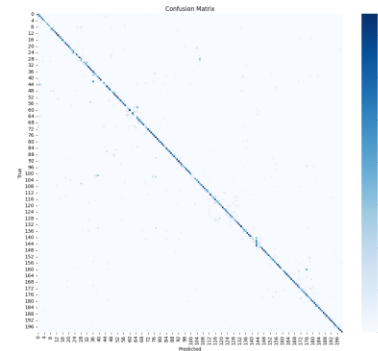
Test accuracy

EfficientNetV2

Learning curves



Confusion matrix



What is Changed?

- label smoothing
- 0.1 -> 0.2
- Label smoothing can prevent overfitting, but increasing value doesn't matter much.

Experiment 3: learning rate 0.0001

[Parameter Setting]

Learning Rate : 0.00001 -> 0.0001

Optimizer : adam

Resize: 384 X 384

Batch size : 32

dropout : 0.5

Tmax = args.epochs

Eta_min: 0.000001

Weight decay: 0.0001

Label smoothing: 0.1

Default EfficientNetV2
Learning rate



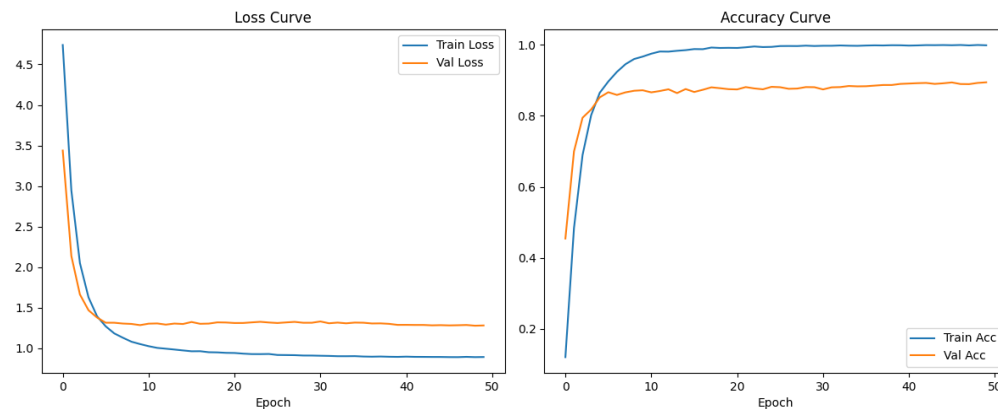
learning rate
0.0001 (multiply 10)

88.3%

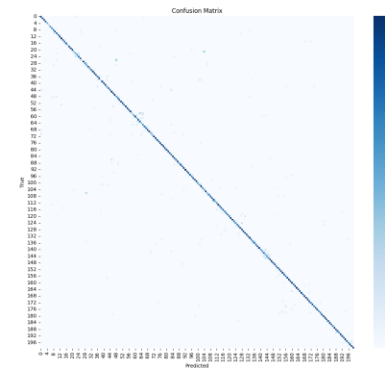
Test accuracy

EfficientNetV2

Learning curves



Confusion matrix



What is Changed?

- Learning rate
- Multiply 10 at default

Why it mattered?

- Fast arrival but Strong Overfitting

Experiment 4: SGD (epoch 70, starting lr 0.0001)

[Parameter Setting]
Learning Rate : 0.0001
Optimizer : adam -> SGD
Resize: 384 X 384
Batch size : 32
Dropout : 0.5
Tmax = args.epochs
Eta_min: 0.000001
Weight decay: 0.0001
Label smoothing: 0.1

Default EfficientNetV2
Optimizer Adam



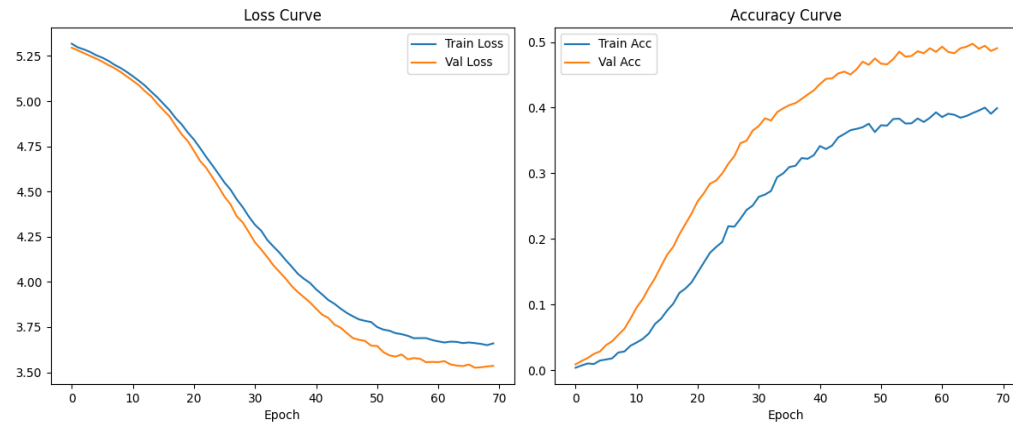
SGD

48.6%

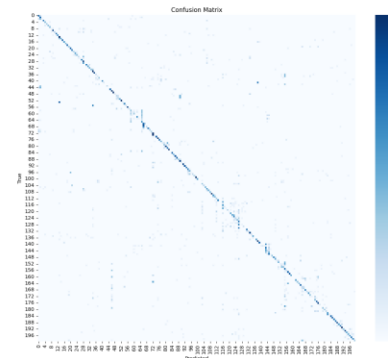
Test accuracy

EfficientNetV2

Learning curves



Confusion matrix



What is Changed?

- Optimizer type (adam -> sgd)

Why it mattered?

- Low accuracy
- Long time of study
- (minimum 1 minute per 1 epoch)

Experiment 5: scheduler

[Parameter Setting]
Learning Rate : 0.00001
Optimizer : adam
Resize: 384 X 384
Batch size : 32
dropout : 0.5
Tmax = args.epochs -> args.epochs/2
Eta_min: 0.000001 -> 0.00001
Weight decay: 0.0001
Label smoothing: 0.1

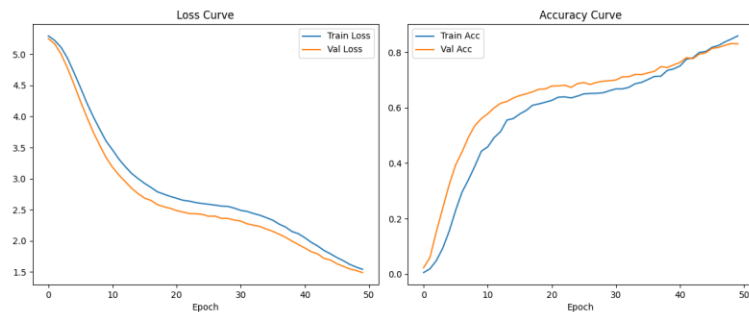
$t_{\max} = \text{args.epochs}$



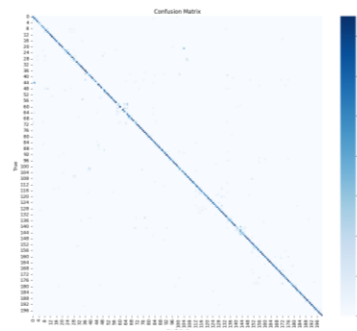
$t_{\max} = \text{args.epochs}/2$

83.4%

Learning curves



Confusion matrix



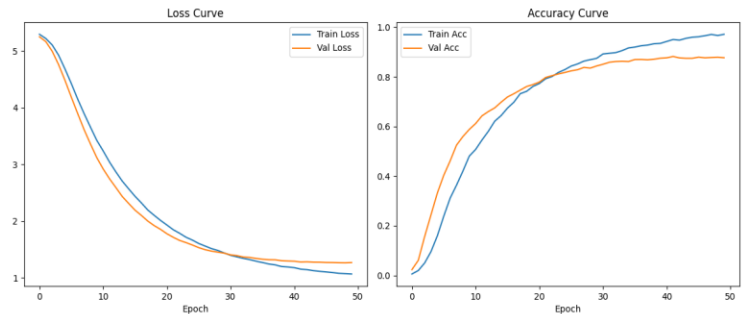
$\text{eta}_{\min} = 0.000001$



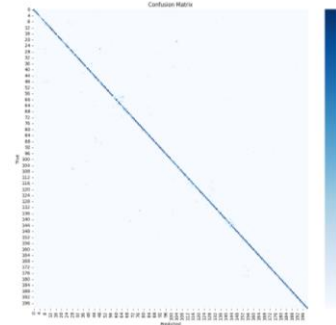
$\text{eta}_{\min} = 0.00001$

87.5%

Learning curves



Confusion matrix



What is Changed?

- $t_{\max} = \text{args.epochs}/2$ -> Allow learning rate to reach its minimum value in half of the total training time. It would converge to peak accuracy faster, but eventually reached peak accuracy in a similar number of epochs.
- $\text{eta}_{\min}$ 10 times -> Raising the minimum threshold of the learning rate to make weight updates more active

Best model combination?

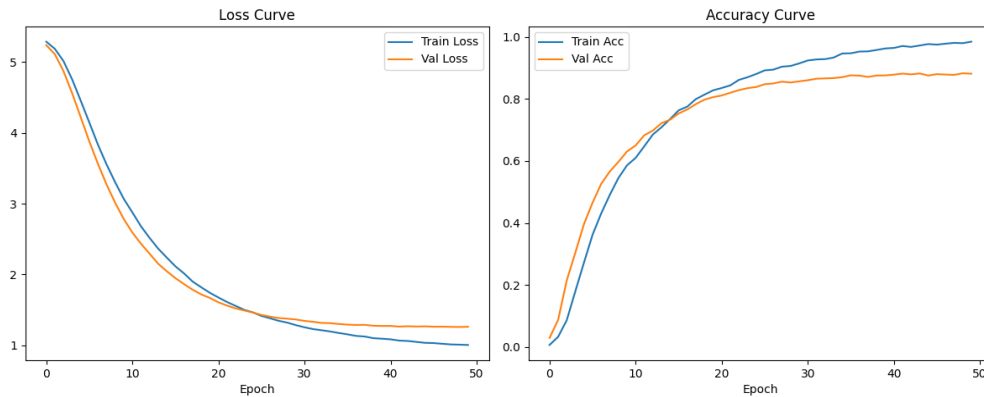
88.13%

Final test accuracy

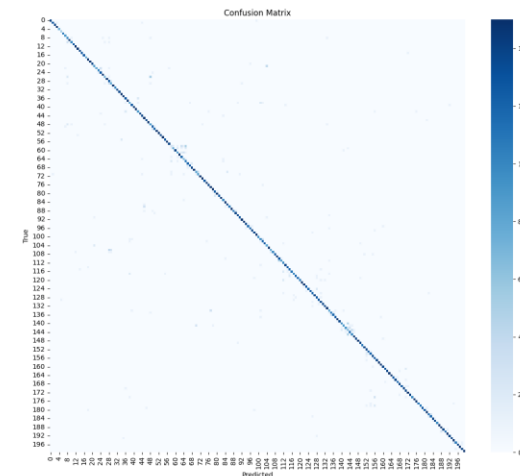
Hyperparameter tuning
combination of the good results... (dropout, eta_min)

[Parameter Setting]
Learning Rate : 0.00001
Optimizer : adam
Resize: 384 X 384
Batch size : 32
dropout : 0.3
Eta_min: 0.00001
Tmax = args.epochs
Weight decay: 0.0001
Label smoothing: 0.1

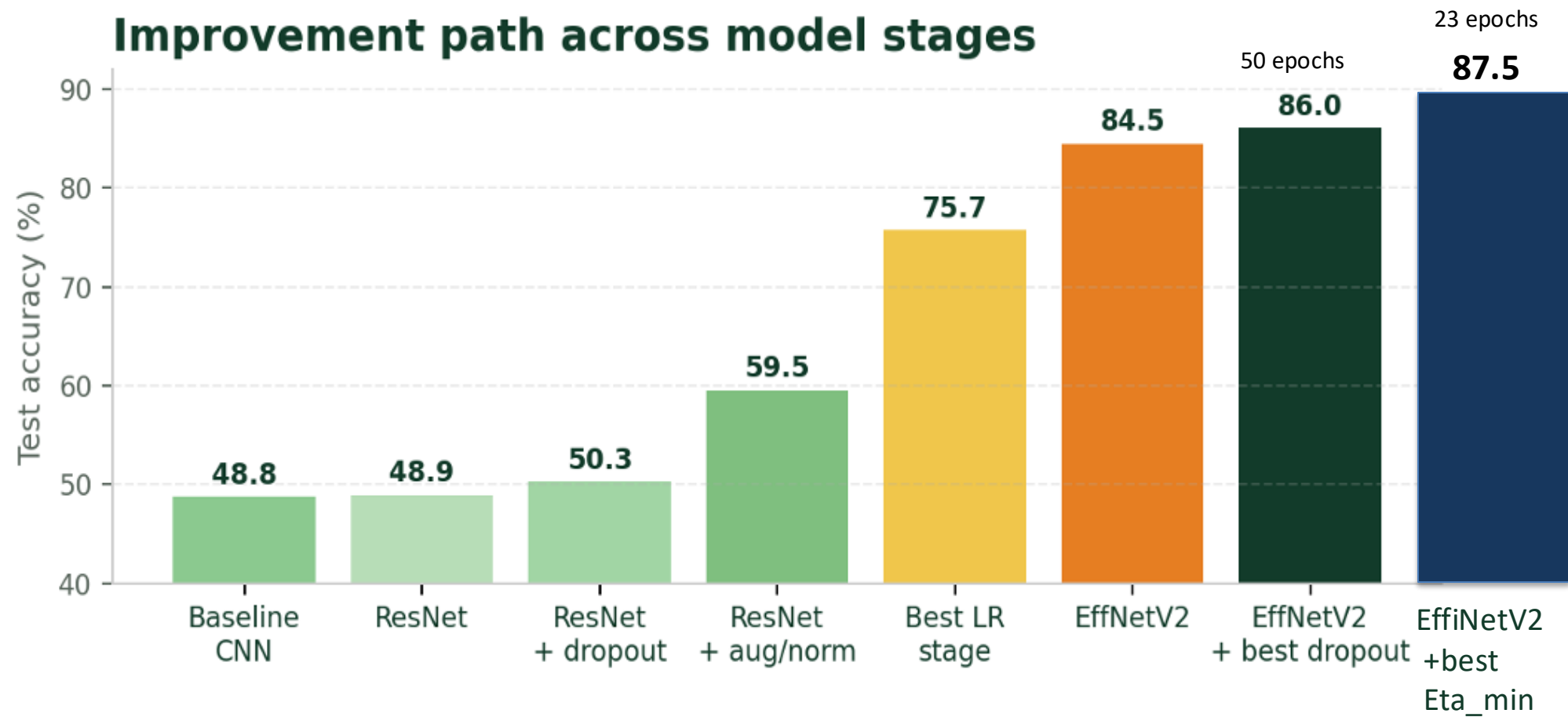
Learning curves



Confusion matrix



Model Comparison Summary



Inference Demo

Streamlit Bird Classifier



Bird Oracle AI

Upload a bird photo. AI will reveal its species with dramatic confidence.

Upload Bird Image

KakaoTa...832431.png
0.9MB



Uploaded Bird

Reveal the Bird

Species Revealed!

Sooty Albatross

[Search this bird on Google](#)

Confidence: 53.24%

AI says: This bird is mysterious...

Top 3 Predictions

1. Sooty Albatross 53.24% Search	2. Brewer Blackbird 6.42% Search	3. Black-footed Albatross 2.52% Search
---	---	---

Limitaion of this project?

- The Trade-off between Accuracy and Overfitting
- Resource & Time Constraints - Hyperparameters
- Advanced Data Augmentation
- Other Improved models
- Problem of dataset

Bird Species Classification Using CNN

Thank you

Group 02 | Eunseo Seo, Minjun Park, Gun Kim, Chaemin Lee

